多模型工作台 项目蓝图与技术规划

Oct 13, 2026 · @Sakura

1. 项目定位与目标

这是一个面向小团队内部的



多模型 AI 工作台：同一个界面里用 OpenAI、Claude、Gemini，让多个 agent 分工协作、调用工具和 MCP，并基于上传文件回答问题；任务既包括通用知识工作，也包括代码任务。后端基于 Microsoft Agent Framework（Python），前后端都由 AI 编写，你只提供提示词。

已确认的决策

事项	结论
使用场景	小团队内部，需要登录和基础权限，不做收费和多租户

事项	结论
接入模型	OpenAI / Azure OpenAI、Anthropic Claude、Google Gemini
第一版功能	多 agent 协作工作流、工具与 MCP 接入、知识库与文件上传
项目起点	从零开始
开发方式	后端 Python，前后端各由一个 AI 会话开发，你只写提示词
任务类型	通用知识工作和代码任务都要
核心分工	Planner–Worker–Reviewer；每个角色的执行者可替换，Reviewer 可由阿里 OpenCodeReview 担任
个性化配置区	与工作区分离；人人可配、可分享；拖拽画布；含试运行、版本历史与回滚、导入导出、角色库与模板

第一版的成功标准（我的假设，请在第 13 节确认）

1. 团队成员登录后，可以在同一个会话里选择、切换模型对话，回答逐字流式显示。
2. 可以创建 agent，把它们编排成工作流，运行时能看到每一步在做什么。
3. 上传文档后，提问的回答附带引用来源，可追溯到原文。
4. 可以挂接 MCP 服务，让 agent 调用外部工具，危险操作需要人工确认。

第一版明确不做

- 对外公开注册、计费、多租户隔离
- 同一问题多模型并排对比（你没有选这项，架构上预留，第二版再做）
- 国内模型和本地模型（你没有选，架构上预留接入位置）
- 移动端原生应用

2. 范围与阶段划分

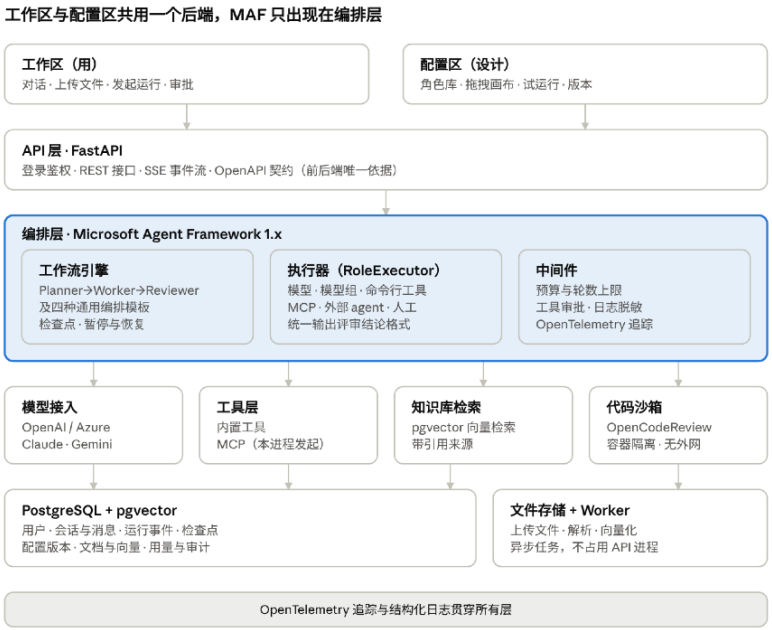
第一版分七个阶段（P0–P6），每个阶段结束都能独立运行、独立验收。多 agent 工作流放在最后，因为它依赖前面的模型接入、工具和知识库。

阶段	内容	依赖
P0 脚手架与契约	仓库结构、CI、mock 模型、登录骨架、接口契约与 OpenAPI 导出	无
P1 多模型对话底座	模型注册表、会话与消息、流式输出、中途切换模型	P0
P2 工具与 MCP	工具注册、MCP 服务器管理、危险操作人工确认	P1
P3 知识库与文件上传	上传、解析、分块、向量化、检索、引用来源	P1
P4 多 agent 工作流引擎	后端工作流引擎、Planner–Worker–Reviewer 核心模板和四种通用编排模板、执行器适配（OpenCodeReview、人工）、代码沙箱、工作区里只读的运行图、暂停/恢复/取消	P2、P3
P5 个性化配置区	角色库、拖拽分工画布、试运行、版本与发布、导入导出、分享	P4
P6 加固与上线	可观测性、成本限额、备份、部署文档、安全检查	P5

基础的多模型对话你没有单独勾选，但它是所有功能的底座，所以放进 P1。P2 和 P3 互不依赖，顺序可以对调。

工作流的编辑方式：按你的选择，直接做可视化拖拽画布，并作为独立的“个性化配置区”与工作区分开（见 5A）。画布的前端工作量大，所以拆成两步：P4 先做后端工作流引擎和官方模板，工作区里只读展示运行图；P5 再做配置区。

3. 总体架构



总体架构 · 5 层，编排层高亮

请求从前端经 API 层进入编排层，由 MAF 调度模型、工具、知识库和代码沙箱，所有状态落在 PostgreSQL；配置区只改配置，不直接执行任务。

一次请求的流程

1. 工作区发起运行，API 层校验登录和预算。
2. 读取已发布的配置版本，用自己的消息表重建历史上下文。
3. 编排层按工作流调度各角色的执行者，过程以 SSE 事件推给前端。
4. 需要审批的工具调用会暂停运行，等你确认后继续。
5. 结果、事件和用量写入数据库。

4. 技术选型

选型原则：小团队、AI 维护，所以优先成熟、AI 熟悉的组件，并尽量用一个 PostgreSQL 解决存储。Microsoft Agent Framework 已在 2026 年 4 月发布 1.0，Python 包名为 agent-framework；其中 DevUI、AG-UI、托管集成、Skills 仍是预览功能，不放进核心路径。

领域	选择	理由	备选
智能体框架	Microsoft Agent Framework 1.x	已定；自带多模型连接器、工作流、MCP、记忆、检查点	—
语言与工具链	Python 3.11+、uv、ruff、pytest、pyright	依赖锁定、检查自动化，适合 AI 自检	mypy
Web 框架	FastAPI + Uvicorn	异步、自动生成 OpenAPI；MAF 的 AG-UI 适配也基于它	Litestar
数据库	PostgreSQL + pgvector	业务数据和向量在同一个库，运维最省	本地开发用 SQLite；规模变大再换独立向量库
异步任务	Postgres 任务表 + 独立 worker 进程	文件解析、向量化耗时，不能占用 API 进程；不额外引入 Redis	Redis 队列
前后端协议	自定义 REST + SSE 契约，事件命名参考 AG-UI	AG-UI 目前是预览；前端由 AI 生成，需要稳定、可测试的契约	直接用 AG-UI + CopilotKit
嵌入模型	OpenAI / Azure OpenAI 的嵌入模型	Anthropic 本身不提供嵌入模型，需要另选；开工前再核对最新型号	Gemini 嵌入
前端	React + TypeScript + Vite + Tailwind + shadcn/ui	AI 最熟悉，类型由 OpenAPI 自动生成，接口不一致会在编译期报错	Vue

领域	选择	理由	备选
登录	邮箱密码 + 服务端会话 Cookie，管理员邀请	小团队最简单	企业 SSO（OIDC）
可观测性	OpenTelemetry + 结构化日志	MAF 内置 OpenTelemetry，并会把追踪上下文传给本地 MCP 服务	—
部署	Docker Compose，单台服务器，Caddy 反向代理	团队规模下足够，易备份、易迁移	云托管容器服务

提醒：MAF 预览期的教程和 1.0 文档写法不完全一致（例如会话的叫法），所以 AI 写代码时必须对照 1.x 的官方文档和 samples，并锁定版本。

5. 核心模块设计

核心思路：业务代码只依赖自己定义的接口，MAF 被限制在“编排层”内，这样框架升级或预览功能变动时，影响范围可控。

5.1 Planner–Worker–Reviewer 核心分工（根据你的补充）

- Planner：把用户任务拆成步骤清单，每步写明负责的 Worker 类型和验收标准，输出结构化的计划。
- Worker：按步骤执行，可调用工具、MCP 和知识库；互不依赖的步骤可并行执行。
- Reviewer：对照验收标准检查每一步和最终结果，给出“通过”或“退回并说明原因”；退回后回到 Worker，默认最多重做 2 轮。
- 汇总：通过后输出最终答案，并附上步骤记录。

实现方式：用 MAF 的图工作流把三个角色连成“规划 → 执行 → 评审 → 退回或通过”，而不是把所有事塞进一个大提示词。三个角色可以各选模型，例如 Planner 和 Reviewer 用较强的模型，Worker 用性价比高的模型，并让 Reviewer 与 Worker 使用不同的模型或提示词，避免“自己给自己打高分”。可配置项：最大轮数、预算上限、规划后是否需要人工确认。另外提供四种通用模板（顺序、并发、交接、群聊），MAF 1.0 已稳定支持这些编排模式。

角色不绑定单个模型（执行器）

每个角色是一个“槽位”，统一由 RoleExecutor 接口承接，槽位里可以换不同的执行器：

执行器	说明	例子	阶段
模型 agent	一个模型加一段指令	Planner 用 Claude，Worker 用 GPT	P4
多模型评审组	多个模型各审一遍再汇总	三家模型各自评审，取交集或投票	P4（可选）
命令行工具适配器	在沙箱里调用命令行工具并解析输出	OpenCodeReview（ocr review）	P4
MCP 工具	通过 MCP 调用外部服务	其他检查、测试服务	P2 之后
外部 agent（A2A）	MAF 支持 A2A 协议，可与其他框架的 agent 协作	预留	之后
人工	暂停等待人工决定	审批、验收	P2

Reviewer 无论由谁担任，都输出统一的结构化结论：是否通过、问题列表（严重程度、位置、说明）、总结。工作流只认这个格式，适配器负责把工具的输出转换成它。技术上，MAF 的工作流既能连接 agent，也能连接普通函数，所以外部工具可以用“函数节点”封装。按任务类型路由：代码任务由 OpenCodeReview 评审（可叠加模型评审），通用任务由模型评审。

关于 OpenCodeReview（按你确认的阿里 alibaba/open-code-review）

- 它是命令行工具，用 npm 安装，依赖 Git 2.41 或更高版本，兼容 OpenAI 和 Anthropic 风格的 API，Apache 2.0 开源。
- 所以接入它需要：一个带 Git 的代码工作区（沙箱）、Node 环境，和模型 API 密钥。
- 它自己也会调用模型，这部分用量和数据外发要纳入预算与审计。
- 它的输出格式我没有核对细节，P4 开始前要先做一次小验证：确认输出能否稳定解析、在沙箱里能否跑通。

5.2 模型接入层

- 模型注册表：厂商、模型名、能力标记（流式、工具调用、图片、上下文长度）、单价、启用状态，由管理员配置，不写死在代码里。
- 通过 MAF 自带的连接器接入三家模型，业务代码只依赖自定义的 ModelGateway 接口。
- 统一超时、重试、限流错误处理；提供 mock 模型，供前端联调和自动测试使用。

5.3 会话与记忆

- 自己的消息表是历史的唯一事实来源，每次运行时再重建给 MAF 的上下文。原因：中途切换模型需要中性格式的历史；我没有验证 MAF 序列化的会话能否跨厂商复用，所以不依赖它。
- 长对话的裁剪或摘要策略放在这一层；跨会话的长期记忆第一版不做，只预留接口。

5.4 工具与 MCP

- 工具注册表包括内置函数工具和 MCP 服务器；MCP 服务器配置包含地址、认证方式、允许调用的工具白名单。
- 每个工具标风险级别（只读、写入、破坏性）；写入和破坏性操作必须经人工确认，用 MAF 的 human-in-the-loop 请求机制实现，工作流运行会暂停等待确认。
- 第一版只用“由我们进程发起调用”的 MCP，不用厂商托管的 MCP。原因：托管 MCP 的调用由模型厂商的服务发出，我们的白名单、审批和审计日志看不到。
- 调用设超时和结果大小上限。

5.5 知识库

1. 上传后校验类型与大小，写入任务表。
2. worker 解析（txt、md、docx、文字型 PDF）、分块、向量化，写入 pgvector。
3. 文件状态：排队、处理中、可用、失败，前端可见。
4. 检索以“搜索知识库”工具的形式提供给 agent，由 agent 决定何时检索；回答附引用（文件名、页码或段落）。
5. 删除文件时同步删除分块；分块记录嵌入模型名，以便以后换模型重建索引。

扫描件 PDF 的 OCR 和复杂表格解析第一版不做。

5.6 运行与状态

- 一次“运行”指一次对话回合或一次工作流执行，状态：排队、运行中、等待确认、成功、失败、已取消。
- MAF 工作流支持检查点和暂停恢复，但我从文档里只看到内存存储和文件存储的示例，没有查到官方的 Postgres 实现，所以 P4 需要自己实现一个数据库支持的检查点存储。
- 支持取消、超时、预算上限中止；刷新页面后可用运行编号重新连上事件流。

5A. 个性化配置区（与工作区分离）

产品分成两个区域：工作区只负责“用”，配置区负责“设计和调整”。两区共用后端，但入口、界面和数据状态分开，改配置不会影响正在运行或已完成的任务。

区域	做什么	谁用
工作区	对话、上传文件、选择已发布的分工、发起运行、查看结果、审批	全体成员
配置区	创建角色、画布编排分工、试运行、版本管理、导入导出、分享	全体成员，各管各的，可分享

配置区的能力

1. **角色库**：创建、复制、分类，也可以从官方模板开始。一个角色包括：名称、职责说明、指令、执行者（单模型、模型组、外部工具、外部服务、人工）、模型、可用工具和知识库、输出格式、预算和轮数上限。
2. **分工画布**：拖拽角色节点并连线，支持顺序、并行、条件分支、评审退回循环和人工确认点；节点上可以随时替换执行者。MAF 1.0 的图工作流本身支持条件分支、并行扩散和汇聚，画布就是它的可视化编辑器。
3. **试运行**：用示例输入在隔离环境里跑，逐步查看输入输出、耗时和费用；不进入正式会话，费用计入运行者的预算。
4. **版本与发布**：改动先保存为草稿，试运行满意再“发布”生成新版本。工作区只使用已发布的版本，可回滚到任一历史版本；每次运行都记录所用的配置版本，历史运行可复现。
5. **导入导出**：配置可导出为文件用于备份和分享。MAF 1.0 支持用 YAML 声明 agent 和工作流，导出格式尽量贴近它，画布布局等信息另存，具体字段 P5 开始前再核对。

6. **分享**：默认私有，可分享给指定成员或全团队，被分享的配置默认只读，可“复制后修改”；管理员可把配置设为官方模板。
7. **发布前校验**：自动检查是否有孤立节点、循环是否设了最大轮数、引用的模型、工具和知识库是否可用、是否设了预算上限。

规则：配置只决定“用什么、怎么分工”，不授予额外权限。工具审批、预算和知识库可见范围永远按“运行者”的权限计算，所以别人分享的配置不能让你越权。

前端工作量提醒：拖拽画布是整个项目前端最重的一块，所以它单独成一个阶段（P5），建议基于成熟的流程图组件库实现，不要让 AI 从零手写画布。Planner–Worker–Reviewer 核心模板在这里就是一份官方模板，可以复制后随意改。

6. 数据模型

数据模型围绕“会话 → 消息 → 运行 → 事件”四层展开，业务数据和向量在同一个 PostgreSQL 里。模型厂商的 API 密钥放在环境变量或密钥管理服务里，不存数据库明文。

表	用途	关键字段
users	账号	email、密码哈希、角色（管理员/成员）、是否禁用
models	模型注册表	厂商、模型名、能力标记、单价、启用状态
conversations	会话	用户、标题、默认模型、可见性
messages	消息（历史的唯一事实来源）	会话、角色、分段内容（文本、工具调用、引用）、模型、所属运行
agents	agent 定义	名称、角色（Planner/Worker/Reviewer/自定义）、执行者类型（单模型/模型组/外部工具/外部服务/人工）、指令、模型、可用工具

表	用途	关键字段
workflows	编排配置	模板类型、参与 agent、最大轮数、预算上限、是否需要确认计划
runs	一次运行	所属会话、编排、状态、起止时间、错误、已用预算
run_events	运行事件流水，可回放	运行、序号、类型、负载
checkpoints	工作流检查点	运行、检查点编号、状态快照
approvals	人工确认请求	运行、请求编号、工具、参数、结果、决定人
tools	工具注册	名称、来源（内置/MCP）、风险级别、参数结构
mcp_servers	MCP 配置	名称、地址、加密的认证信息、工具白名单、启用状态
knowledge_bases	知识库	名称、可见范围、嵌入模型
documents	文件	所属知识库、文件名、大小、存储路径、处理状态、错误
chunks	分块与向量	所属文件、序号、文本、向量、页码或段落位置、嵌入模型
jobs	异步任务	类型、负载、状态、重试次数
usage_logs	用量与成本	运行、模型、输入/输出 token、费用、用户
audit_logs	审计	用户、动作、对象、时间
config_versions	配置区的角色与分工（草稿与已发布版本）	配置编号、版本号、状态（草稿/已发布）、角色与画布内容、发布人；runs 记录所用配置版本，试运行用 is_test 标记

表	用途	关键字段
shares	配置分享	配置编号、分享对象（成员/全团队）、权限（只读）、是否官方模板

7. 接口与前后端契约

前后端只通过本节和自动导出的 OpenAPI 对齐：所有路径以 /api/v1 开头，前端据 OpenAPI 自动生成 TypeScript 类型。约定：字段用 snake_case，时间用 UTC 的 ISO 8601，ID 用 UUID，列表用游标分页；改接口先改契约再改代码。

接口清单（这是第一版草案，P0 阶段由 AI 细化）

资源	接口	说明
认证	POST /auth/login、POST /auth/logout、GET /auth/me	登录、退出、当前用户
模型	GET /models	可用模型及能力标记
会话	GET/POST /conversations、GET/PATCH/DELETE /conversations/{id}、GET /conversations/{id}/messages	会话与历史消息
运行	POST /conversations/{id}/runs	启动一次运行，以 SSE 返回事件
运行	GET /runs/{id}/events?after=序号	断线后续传事件
运行	POST /runs/{id}/cancel	取消运行
运行	POST /runs/{id}/approvals/{request_id}	对工具调用或计划做“同意/拒绝”
agent	CRUD /agents	agent 管理
编排	CRUD /workflows	编排配置

资源	接口	说明
工具与 MCP	GET /tools、CRUD /mcp-servers、POST /mcp-servers/{id}/test	工具列表、MCP 配置、连通性测试
知识库	CRUD /knowledge-bases、POST /knowledge-bases/{id}/documents、GET/DELETE 文件	上传、状态、删除
用量	GET /usage	按时间和用户查询用量与费用
管理	GET/POST /admin/users	邀请和禁用成员
配置区·角色	CRUD /studio/roles	角色库
配置区·分工	CRUD /studio/workflows、POST /studio/workflows/{id}/publish、GET /studio/workflows/{id}/versions、POST /studio/workflows/{id}/rollback	草稿、发布、版本、回滚
配置区·试运行	POST /studio/workflows/{id}/test-runs	试运行，以 SSE 返回事件
配置区·导入导出与分享	GET /studio/workflows/{id}/export、POST /studio/import、POST /studio/workflows/{id}/shares	导出、导入、分享

流式事件（SSE）

每个事件带递增的序号，前端据此断线重连。

事件	含义	主要字段
run.started	运行开始	run_id、编排模板
plan.created	Planner 产出计划	步骤列表、验收标准
step.started / step.completed	某一步开始或结束	step_id、角色、执行者
message.delta	文本片段	message_id、text
message.completed	一条消息结束	message_id
tool.call.started / completed	工具调用	工具名、参数摘要、结果摘要

事件	含义	主要字段
approval.requested	需要人工确认	request_id、工具、参数、风险级别
review.result	Reviewer 的评审结论	结论、问题清单（严重程度、位置、说明、建议）
citations	引用来源	文件、位置、片段
usage	用量	token、费用
run.failed / run.completed	结束	状态、错误码

错误格式

统一为 error（code、message、request_id）。错误码：unauthorized、forbidden、not_found、validation_failed、model_unavailable、rate_limited、budget_exceeded、run_conflict、tool_denied、internal_error。前端必须为每个错误码准备人能看懂的提示。

8. 安全与权限

最大的风险不在登录，而在“模型被不可信内容诱导去调用工具”和“文档数据流向第三方”。

- **账号**：管理员邀请制，密码用 argon2 哈希，会话 Cookie 设为 HttpOnly 和 SameSite，登录限流；角色只有管理员和成员。
- **密钥**：模型厂商密钥和 MCP 凭据只存在服务端，前端永远接触不到；日志必须脱敏；定期轮换。
- **工具调用**：白名单加风险分级，写入和破坏性操作默认需要人工确认；来自文档、网页、工具结果的内容一律当作不可信数据，其中的“指令”不能提升权限；工具只拿最小权限的凭据。
- **代码与外部工具任务**：在沙箱容器里运行，限制网络、资源和运行时间，工作目录按任务隔离。
- **文件**：类型白名单和大小上限，解析放在 worker 进程，存储路径不由用户决定，下载要鉴权。
- **知识库隔离**：检索时按可见范围过滤，避免一个人搜到别人的私有文件。
- **数据外发**：上传的内容和对话会发送给所选模型厂商，界面上要标明；管理员可以限制某个知识库只能用指定模型。

- **审计**：登录、工具审批、文件上传和删除、配置变更都写入审计日志。

9. 部署与运维

第一版部署在单台服务器上，用 Docker Compose 一条命令起停，重点放在“可观测”和“花钱可控”上。

- **环境**：开发环境用 mock 模型加本地 Postgres；CI 跑测试和契约检查；生产环境由反向代理（Caddy）、API、worker、Postgres（含 pgvector）、前端静态文件和代码沙箱容器组成。
- **配置**：用环境变量管理，提供 .env.example；密钥不进仓库、不进数据库明文。MAF 不会自动读取 .env，程序启动时要显式加载。
- **可观测性**：开启 MAF 自带的 OpenTelemetry 追踪，一次运行里的模型调用、工具调用、工作流步骤都能串成一条链；结构化日志带 run_id 和 user_id，并脱敏；指标包括请求量、延迟、错误率、token 和费用。
- **成本控制**：每人每日、每月预算，每次运行的预算上限；工作流最大轮数（Planner–Worker–Reviewer 的退回循环和群聊最容易失控）；模型白名单；超过预算 80% 时提醒，超过 100% 自动中止运行。
- **备份**：Postgres 每日备份，上传文件目录同步备份，定期做一次恢复演练。
- **升级**：依赖全部锁定版本；MAF 升级单独提交，并跑完回归用例再合并。
- **沙箱**：代码任务在独立容器里运行，限制 CPU、内存和运行时间，默认不开放外网，只允许访问必要的地址。
- **网络**：服务器必须能稳定访问所选模型厂商的 API，这一点在第 12 节单独说明。

10. AI 协作开发规范

前后端两个 AI 会话不跑偏的关键，是它们共享同一组文件作为事实，而不是靠各自的聊天记忆。

文件	作用	谁维护
AGENTS.md（后端、前端各一份）	工作规则、技术约束、汇报方式	你确认，AI 按需更新
docs/design.md	设计文档，即本蓝图的落地版	后端 AI

文件	作用	谁维护
docs/api-contract.md 和导出的 OpenAPI	前后端唯一的对齐依据	后端 AI 改，前端 AI 只读
docs/decisions.md	每个技术决策和理由	两边都可追加
docs/PROGRESS.md	已完成、进行中、待办、已知问题	每个 AI 会话结束前更新

工作方式

- 契约先行：后端先交 OpenAPI 和 mock 模式，前端据此自动生成 TypeScript 类型并开发，不等真实模型。
- 小步前进：每个阶段拆成若干小任务，一个任务一次提交，不跨阶段。
- 自动门禁：后端需要 ruff、pyright、pytest 和“导出的 OpenAPI 与契约一致”的检查；前端需要类型检查、代码检查和基本的端到端烟雾测试。门禁不过不算完成。
- 新会话开工前先读 AGENTS.md、docs/design.md 和 docs/PROGRESS.md。
- 每条提示词包含四点：目标、范围、约束、验收标准；验收标准尽量写成“你可以点击哪里、看到什么”。

把 Planner–Worker–Reviewer 用在开发过程上（我的建议，可选）：同样的分工也适合用来开发这个项目本身。一个 AI 会话拆任务、写验收标准，另一个写代码，再用 OpenCodeReview 加另一个模型审查代码改动。审查和写代码的不要是同一个会话，否则容易自我确认。

11. 里程碑与验收标准



路线图 · 7 个阶段，每阶段一道验收关

总计粗估 11 到 14 周。AI 写代码很快，真正的瓶颈通常是你验收和反馈的节奏，所以每个阶段都给出你可以亲手点一遍的检查步骤。

阶段	你可以这样检查
P0	打开前端页面，选 mock 模型发一句话，看到逐字输出；打开接口文档页，能看到全部接口
P1	分别选三家模型发消息；同一会话中途切换模型继续聊；刷新页面后历史还在
P2	添加一个 MCP 服务并点“测试”；让 agent 调用一个只读工具；再触发写入类工具，应弹出确认，拒绝后不执行
P3	上传一个 PDF，等状态变为“可用”；提问，回答带引用，点击引用能看到原文位置；删除文件后再问，不再引用它

阶段	你可以这样检查
P4	在工作区选 Planner–Worker–Reviewer 提交任务，依次看到计划、执行、评审和退回；中途暂停再恢复；代码任务由 OpenCodeReview 给出问题清单
P5	在配置区新建两个角色，在画布上连成分工；试运行并查看每步输入输出和费用；发布后在工作区选用；回滚到上一个版本；导出再导入
P6	把预算调得很低跑一次，应自动中止；从备份恢复一次；能在追踪里看到一次运行的完整链路

12. 你可能没考虑到的点

最需要先确认的是第 1 项“服务器位置与可达性”，它会直接决定三家模型能不能用。

要点	为什么重要	建议做法
1. 服务器位置与可达性	OpenAI、Anthropic、Google 对服务地区有限制，而你的时区显示在中国大陆；服务器所在地不在支持范围内，调用会失败，也可能违反使用条款	先确认团队和服务器所在地，以各厂商最新的支持地区列表为准；服务器放在支持的地区，或改用云厂商在当地合规提供的模型服务；不要用不合规的绕行方式
2. 数据外发与合规	上传的文档和对话会发送给第三方模型厂商，MAF 官方也提醒使用者自行评估这类数据流向	上传前明确提示；可按知识库限制允许使用的模型；约定禁止上传的敏感资料类型
3. 成本叠加失控	Planner–Worker–Reviewer 的退回循环、群聊，以及 OpenCodeReview 自己也调用模型，费用会一层层叠加	每次运行设预算和最大轮数，超限中止；角色分级选模型

要点	为什么重要	建议做法
4. 提示词注入	文档、网页、工具返回的内容里可能夹带“指令”，诱导模型调用工具	见第 8 节：当作不可信数据、最小权限、写入操作人工确认
5. Reviewer 的质量无从衰量	同一模型评自己的输出容易放水，工具也有误报和漏报	评审和执行用不同的模型或工具；准备一组“已知有问题”的样例，检验 Reviewer 能否发现
6. 缺少评测集	没有固定的检验方法，就不知道换模型、升级 MAF 、改提示词是变好还是变差	P3 起建一组固定的知识库问答题，P4 起建工作流回归用例，每次大改动前后都跑一遍
7. 嵌入模型绑定	换嵌入模型需要重建全部索引，而且 Anthropic 本身不提供嵌入模型	先固定一个，分块记录所用模型，为将来重建留好接口
8. 中文检索质量	纯向量检索对专有名词、编号召回差，且 Postgres 默认的全文检索对中文需要额外扩展	第一版用纯向量，用评测集看效果，再决定要不要加关键词检索
9. 中途切换模型	不同厂商的工具调用格式、图片支持不同	历史以自己的消息表为准，切换前检查目标模型的能力标记
10. MAF 版本与预览功能	预览期教程和 1.0 文档不一致，DevUI、AG-UI 等仍是预览	锁定版本，预览功能不进核心路径，升级走回归测试
11. 长任务的可靠性	取消、超时、服务重启后恢复；官方检查点示例只看到内存和文件存储	自己实现数据库检查点，运行事件持久化，重启后能恢复或明确报失败
12. 前端 AI 开发的常见遗漏	空状态、加载中、错误提示、断线重连、长文本和 Markdown、代码高亮、手机适配	写进前端 AGENTS.md 的验收清单
13. 人员变动	成员离开后账号和密钥的处理常被忽略	禁用账号，并轮换其接触过的密钥

13. 待你确认的问题

第 1 项是阻塞项，其余你不回答时我会按“我的推荐”一列推进。

问题	我的推荐	影响
1. 团队和服务器在哪里?	先告诉我，我再给具体方案	决定三家模型能不能用、服务器放哪
2. 登录方式	邮箱密码加管理员邀请；企业 SSO 以后再加	SSO 会增加 P0 的工作量
3. 会话和知识库的可见范围	会话默认私有，知识库可设为团队共享	影响数据隔离和检索过滤
4. 每人每月预算上限	先设一个保守值，管理员可调	影响限额和告警逻辑
5. 可上传的文件	txt、md、docx、文字型 PDF，单文件 20 MB 以内	影响解析工作量
6. 代码任务的代码从哪里来	第一版支持上传压缩包或只读克隆仓库，不做写回和提 PR	影响沙箱和权限设计
7. Planner 的计划是否需要你确认后才执行	默认需要，可在编排里关闭	影响体验和成本
8. 第一批要接入的 MCP 服务	请列出 1–3 个	决定 P2 的验收对象
9. GPT 对话中的想法	我读不到，请把关键片段粘贴给我	可能改变设计的某些部分